

SecOps Bot Alarm Report

Zero-Trust Vault DevSecOps Project

Detailed report about the alerting bot used for SIEM monitoring,
pipeline failure analysis, and proactive alarms.

Bot source file: `secops-bot/bot.py`

Runtime: Python container

Alert channel: Telegram

Log source: Elasticsearch / Filebeat

CI/CD source: GitLab API

AI analysis: OpenRouter-compatible LLM API

Academic Year 2025–2026

Contents

Executive Summary	1
1 Bot Purpose and Scope	2
1.1 Purpose	2
1.2 Scope	2
1.3 Why the Bot Is Useful	2
2 Bot Architecture	3
2.1 Main Components	3
2.2 High-Level Data Flow	3
3 Configuration	5
3.1 Environment Variables	5
3.2 Example Environment File	5
3.3 Security Note	6
4 Telegram Commands	7
4.1 /start	7
4.2 /siem	7
4.3 /pipelines	8
5 Proactive Alarming	9
5.1 Background Monitoring	9
5.2 Alarm Detection Logic	9
5.3 Alarm Threshold	10
5.4 Alert Message	10
5.5 Alarm Use Case	10
6 LLM Analysis Layer	12
6.1 Fallback Model Chain	12
6.2 Value of AI Summarization	12
6.3 Limitations	13

7	Operational Workflow	14
7.1	Deployment	14
7.2	How to Demonstrate the Bot	14
7.3	Suggested Screenshot	15
8	Risks and Improvements	17
8.1	Current Risks	17
8.2	Recommended Improvements	17
9	Conclusion	19
A	Compilation	20

List of Figures

7.1 SecOps bot alarming workflow	15
--	----

List of Tables

2.1	Bot architecture components	3
3.1	Bot environment variables	5
8.1	Bot-specific risks	17

Executive Summary

This report focuses only on the SecOps bot used in the DevSecOps project. The bot is designed as an operational assistant for alarming and incident awareness. It connects to Telegram for communication, Elasticsearch for runtime security logs, GitLab for CI/CD pipeline status, and an LLM provider for automatic analysis of logs and failures.

The main purpose of the bot is to reduce the time between detection and response. Instead of requiring a human operator to manually check Kibana dashboards and GitLab jobs, the bot provides simple commands and proactive background monitoring. When suspicious events appear, especially repeated unauthorized access responses, the bot sends an alert directly to Telegram with a short AI-generated incident summary.

Chapter 1

Bot Purpose and Scope

1.1 Purpose

The SecOps bot was created to support the operational part of the DevSecOps lifecycle. CI/CD pipelines detect many issues before deployment, but some problems only appear at runtime. The bot helps monitor those runtime events and turns raw logs into understandable alerts.

1.2 Scope

The bot covers three main operational needs:

- Manual SIEM analysis through the `/siem` Telegram command.
- Manual GitLab pipeline failure analysis through the `/pipelines` Telegram command.
- Proactive alarming through a scheduled background job that detects suspicious authentication failures.

1.3 Why the Bot Is Useful

The bot is useful because it centralizes three different sources of information:

1. Runtime logs from Elasticsearch.
2. CI/CD job status and traces from GitLab.
3. Human-readable incident explanations generated through an LLM.

This gives the security operator a faster way to understand what is happening without switching between many tools.

Chapter 2

Bot Architecture

2.1 Main Components

Table 2.1: Bot architecture components

Component		Role
Telegram Bot API		Provides the chat interface used by the operator.
Elasticsearch Client		Queries runtime logs collected by Filebeat.
GitLab Client		Reads failed pipelines and job traces from GitLab.
OpenRouter / LLM Client		Summarizes logs and CI/CD failures in natural language.
Background Queue	Job	Runs periodic checks for suspicious events.
Environment	Variables	Stores tokens, URLs, and sensitive configuration.

2.2 High-Level Data Flow

The bot receives commands or scheduled events, queries external systems, sends the collected evidence to the LLM for analysis, and returns the result through Telegram.

1. The operator starts the bot with `/start`.
2. The bot stores the Telegram chat ID for future alerts.
3. The operator can request a SIEM check using `/siem`.
4. The operator can request pipeline status using `/pipelines`.
5. A background task periodically checks recent Elasticsearch logs.

6. If suspicious events are detected, the bot sends a proactive alert to Telegram.

Chapter 3

Configuration

3.1 Environment Variables

The bot is configured through environment variables. This avoids storing real tokens directly in the code.

Table 3.1: Bot environment variables

Variable	Purpose
TELEGRAM_TOKEN	Telegram bot token used to start the bot.
TELEGRAM_CHAT_ID	Default chat ID for background alerts.
OPENROUTER_API_KEY	API key used to call the LLM provider.
GITLAB_URL	GitLab instance URL.
GITLAB_TOKEN	GitLab private token used to read projects and pipelines.
ELASTICSEARCH_URL	Elasticsearch endpoint used to query logs.

3.2 Example Environment File

```
1 TELEGRAM_TOKEN=replace-me
2 TELEGRAM_CHAT_ID=replace-me
3 OPENROUTER_API_KEY=replace-me
4 GITLAB_URL=https://gitlab.com
5 GITLAB_TOKEN=replace-me
6 ELASTICSEARCH_URL=http://elasticsearch:9200
```

Listing 3.1: Bot environment example

3.3 Security Note

The real `.env` file must not be committed to Git. It contains secrets that can give access to Telegram, GitLab, and the LLM API. In production, these values should be stored in GitLab CI/CD protected variables, Docker secrets, Kubernetes secrets, or a dedicated secret manager.

Chapter 4

Telegram Commands

4.1 /start

The `/start` command initializes the bot conversation. It also binds the current Telegram chat ID so the bot knows where to send proactive alarms.

```
1 async def start(update: Update, context: ContextTypes.  
    DEFAULT_TYPE):  
2     global ALERT_CHAT_ID  
3     ALERT_CHAT_ID = update.effective_chat.id  
4     welcome_msg = "SecOps AI Agent Online..."  
5     await update.message.reply_text(welcome_msg, parse_mode='  
        Markdown')
```

Listing 4.1: Start command behavior

4.2 /siem

The `/siem` command queries Elasticsearch for recent suspicious logs. The current query searches for log messages containing `401` or `error`. These terms are useful for detecting authentication failures and application problems.

```
1 query = {  
2     "query": {  
3         "bool": {  
4             "should": [  
5                 {"match": {"message": "401"}},  
6                 {"match": {"message": "error"}}  
7             ],  
8             "minimum_should_match": 1  
9         }  
    }
```

```
10     },
11     "size": 5,
12     "sort": [{"@timestamp": {"order": "desc"}}]
13 }
```

Listing 4.2: SIEM query logic

If no suspicious events are found, the bot replies that the SIEM is quiet. If events are found, the bot sends the log messages to the LLM and returns an analysis to Telegram.

4.3 /pipelines

The `/pipelines` command checks GitLab for failed pipelines. If a failed pipeline exists, the bot retrieves the failed job trace and sends the last part of the trace to the LLM.

The expected response is a practical explanation of why the job failed and what commands or changes may fix it.

Chapter 5

Proactive Alarming

5.1 Background Monitoring

The most important alarm feature is the background monitoring job. It runs automatically after the bot starts.

```
1 job_queue = app.job_queue
2 job_queue.run_repeating(monitor_background, interval=10, first=5)
```

Listing 5.1: Background monitor schedule

This means the bot starts checking after five seconds and repeats the check every ten seconds.

5.2 Alarm Detection Logic

The background monitor searches for recent 401 `Unauthorized` events in Elasticsearch.

```
1 query = {
2     "query": {
3         "bool": {
4             "must": [{"match": {"message": "401"}}],
5             "filter": [{"range": {"@timestamp": {"gte": "now-2m"
6                 }}}]
7         }
8     },
9     "size": 5,
10    "sort": [{"@timestamp": {"order": "desc"}}]
```

Listing 5.2: Alarm detection query

The time window is the last two minutes. The bot retrieves the latest five matching logs and checks the total number of hits.

5.3 Alarm Threshold

The current threshold is:

```
1 if res['hits']['total']['value'] > 1:
2     # trigger alert
```

Listing 5.3: Alarm threshold

This means the bot sends an alarm when more than one unauthorized event is detected during the recent time window. The code comment mentions more than three attempts, but the implemented condition is currently more than one. This should be corrected if the intended threshold is three failed attempts.

5.4 Alert Message

When the threshold is reached, the bot asks the LLM to produce a three-bullet incident report. Then it sends the result to Telegram.

```
1 prompt = (
2     "URGENT: I am detecting a spike in 401 Unauthorized errors "
3     "in my SIEM right now. Analyze these logs and give me a "
4     "3-bullet-point incident report:\n"
5 )
6
7 msg = "PROACTIVE THREAT ALERT\n\n" + analysis
8 await context.bot.send_message(
9     chat_id=ALERT_CHAT_ID,
10    text=msg,
11    parse_mode='Markdown'
12 )
```

Listing 5.4: Proactive alert message

5.5 Alarm Use Case

A typical alarm scenario is:

1. An attacker or test user repeatedly sends wrong credentials to the backend login endpoint.

2. The backend returns multiple **401 Unauthorized** responses.
3. Docker writes the backend logs.
4. Filebeat collects the container logs.
5. Elasticsearch indexes those events.
6. The bot background job detects the recent **401** spike.
7. The bot sends the logs to the LLM for summarization.
8. Telegram receives a proactive threat alert.

Chapter 6

LLM Analysis Layer

6.1 Fallback Model Chain

The bot defines a list of LLM models. It tries each model in order until one succeeds. If one model fails, the bot logs the error and continues to the next model.

```
1 MODELS = [  
2     "meta-llama/llama-3.3-70b-instruct:free",  
3     "nvidia/nemotron-3-super-120b-a12b:free",  
4     "openai/gpt-oss-120b:free",  
5     "google/gemma-3-27b-it:free",  
6     "nousresearch/hermes-3-llama-3.1-405b:free",  
7     "openrouter/free",  
8 ]
```

Listing 6.1: Model fallback list

6.2 Value of AI Summarization

Raw SIEM logs and CI/CD traces can be difficult to read quickly. The LLM layer transforms raw evidence into a short explanation. This is useful for:

- Understanding the threat level.
- Identifying the probable cause.
- Suggesting first response actions.
- Summarizing failed CI/CD jobs for faster debugging.

6.3 Limitations

The LLM should be treated as an assistant, not as the final authority. Its output must be verified against the original logs and pipeline traces. Sensitive data should also be filtered before being sent to any external model provider.

Chapter 7

Operational Workflow

7.1 Deployment

The bot runs as a Docker service inside the project stack. It depends on the backend and Elasticsearch services.

```
1 secops-bot:
2   build:
3     context: ./secops-bot
4     dockerfile: ../docker/secops-bot.Dockerfile
5   image: zero-trust-vault/secops-bot:semester
6   container_name: secops-bot
7   env_file:
8     - ./secops-bot/.env.example
9   depends_on:
10    - backend
11    - elasticsearch
12   restart: unless-stopped
```

Listing 7.1: Docker Compose bot service

7.2 How to Demonstrate the Bot

To demonstrate the bot during a presentation:

1. Start the full stack with Docker Compose.
2. Open Telegram and send `/start` to the bot.
3. Generate failed login attempts against the backend.
4. Wait for the background monitor to detect recent 401 events.

5. Show the proactive Telegram alert.
6. Use `/siem` to manually query recent events.
7. Use `/pipelines` after a failed GitLab pipeline.
8. Add the final screenshot as `screenshots/09-secops-bot-alarm-workflow.png`.

7.3 Suggested Screenshot

The required picture should show the bot in action. A good screenshot can include:

- The Telegram chat with the `/start` command.
- A `PROACTIVE THREAT ALERT` message.
- The AI-generated incident summary.
- Kibana or Elasticsearch logs showing matching `401` events.

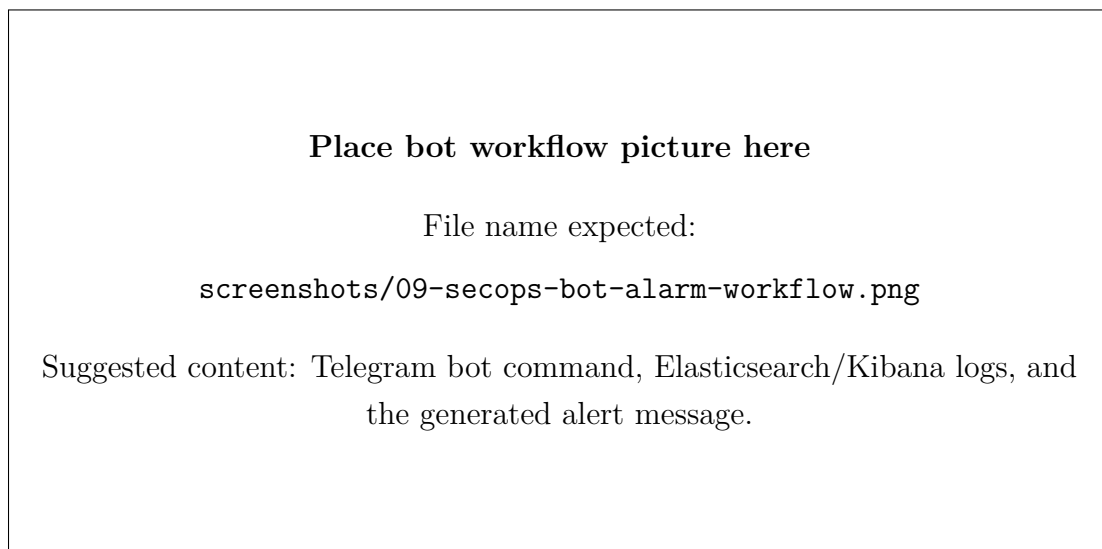
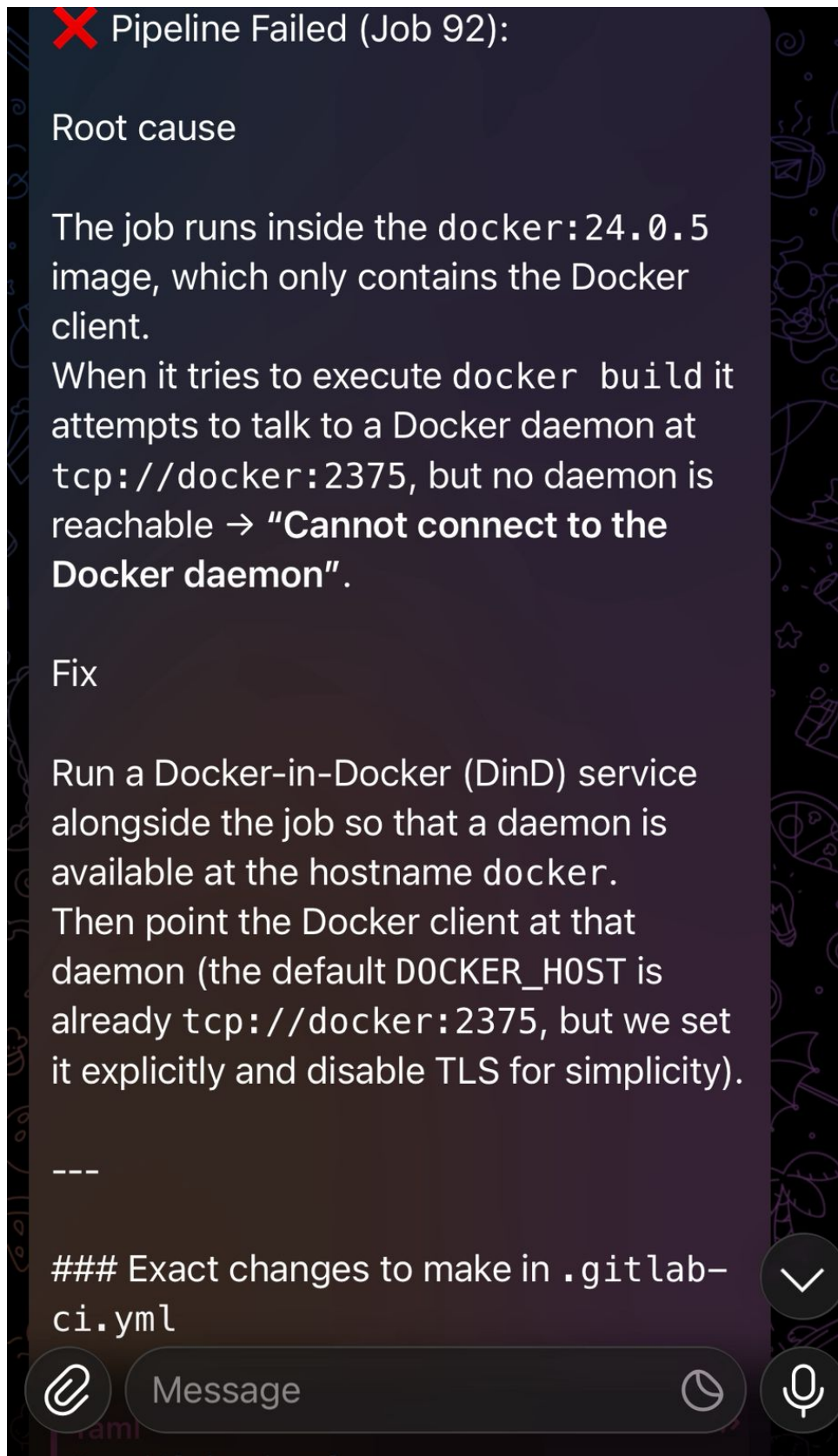


Figure 7.1: SecOps bot alarming workflow



 Pipeline Failed (Job 92):

Root cause

The job runs inside the `docker:24.0.5` image, which only contains the Docker client.

When it tries to execute `docker build` it attempts to talk to a Docker daemon at `tcp://docker:2375`, but no daemon is reachable → **"Cannot connect to the Docker daemon"**.

Fix

Run a Docker-in-Docker (DinD) service alongside the job so that a daemon is available at the hostname `docker`.

Then point the Docker client at that daemon (the default `DOCKER_HOST` is already `tcp://docker:2375`, but we set it explicitly and disable TLS for simplicity).

Exact changes to make in `.gitlab-ci.yml`

Chapter 8

Risks and Improvements

8.1 Current Risks

Table 8.1: Bot-specific risks

Risk	Impact	Mitigation
Sensitive tokens in environment files	Token exposure can compromise Telegram, GitLab, or LLM access.	Use secret managers and never commit real <code>.env</code> files.
External LLM processing	Logs may contain sensitive data.	Redact logs before sending them to the LLM.
Low alert threshold	More than one 401 can cause noisy alerts.	Tune the threshold and add rate limiting.
No alert deduplication	The same incident may produce repeated messages.	Store recent alert fingerprints and suppress duplicates.
Single chat destination	Alerts may go to only one operator.	Support multiple authorized chat IDs or team channels.
Broad GitLab project selection	The bot selects the first owned project.	Configure a specific project ID.

8.2 Recommended Improvements

- Make the unauthorized access threshold configurable.
- Add a cooldown period to avoid repeated alerts for the same incident.

- Configure a fixed GitLab project ID instead of selecting the first owned project.
- Add role-based access so only authorized Telegram users can run commands.
- Redact secrets, tokens, and personal data before sending logs to the LLM.
- Store alert history in a small database or Elasticsearch index.
- Add more detection rules, such as repeated 500 errors, container restarts, and suspicious paths.

Chapter 9

Conclusion

The SecOps bot is the alarming and operational assistant of the DevSecOps project. It connects runtime monitoring, CI/CD visibility, and AI-based summarization into one Telegram interface. The most important feature is proactive alerting: the bot checks Elasticsearch for recent unauthorized access events and sends a Telegram alert when suspicious activity is detected.

This bot turns the project from a simple CI/CD demonstration into a more complete DevSecOps workflow. It shows how teams can detect runtime problems, summarize incidents, and react faster after deployment. With stronger secret handling, alert tuning, access control, and log redaction, the bot could evolve into a useful lightweight security operations assistant.

Appendix A

Compilation

Compile this report with:

```
1 cd devsecops-project/docs
2 pdflatex secops_bot_alarm_report.tex
3 pdflatex secops_bot_alarm_report.tex
```

Listing A.1: Compile the bot report

The screenshot placeholder will automatically use this file if it exists:

```
1 devsecops-project/screenshots/09-secops-bot-alarm-workflow.png
```

Listing A.2: Expected bot workflow screenshot